

GridPulse F1 Replay Analytics

Interview Preparation — Technical Deep Dive

May 14, 2026

Repository analyzed	F:\F1Project
Backend	FastAPI, FastF1, pandas, uvicorn
Frontend	React, Vite, D3, Recharts, Tailwind CSS
Primary challenge	Converting heavy, irregular Formula 1 telemetry into a synchronized browser replay.

TABLE OF CONTENTS

1. Project Overview	3
2. Full Tech Stack Breakdown	4
3. System Architecture	6
4. Key Technical Decisions	9
5. Deep Dive - Hardest Problems	11
6. Interview Questions and Answers	13
7. Follow-up Questions	17
8. Quick Reference Card	19

1. Project Overview

GridPulse is a full-stack **Formula 1 replay analytics** application. It lets a user choose an F1 season and Grand Prix, then replays a race using synchronized **position telemetry**, **car telemetry**, **lap timing**, **strategy**, **weather**, **track status**, **race control**, and **team radio** streams. The product problem is that raw race data is spread across multiple high-volume timing feeds; GridPulse turns that data into a readable post-race terminal for analysis and interview demo use.

The real-world domain is motorsport data analysis. In practice, a race engineer, analyst, fan, or student can inspect how race order, tyre strategy, pit stops, sector performance, weather, and control messages evolved over time. The frontend intentionally uses a dense terminal layout instead of a marketing page, matching the repeated-workflow nature of race review.

Key outcomes and metrics found in the repo

- The README lists live replay, leaderboard gaps, tyre strategy, pit-stop analysis, sector comparisons, race control, team radio, telemetry charts, and final podium presentation as core outcomes.
- The checked-in **Abu Dhabi 2025** export contains **20** race-result rows, **1,156** lap rows, **107** race-control messages, **154** weather rows, **7** track-status rows, and **104,349** exported 1 Hz telemetry rows.
- Prior runtime verification in the source notes and code path targets a full replay response with **telemetry**, **drivers**, **laps**, **events**, **race_control**, **circuit_info**, **weather**, **total_laps**, and **time_base**.
- The backend exposes a lightweight **telemetry_replay_meta** endpoint so a deployment can confirm race shape before downloading the full replay payload.
- The current frontend `.env` points **VITE_API_URL** at a Hugging Face Space backend, while local development defaults to **http://localhost:8000**.

Evidence base read from the project

- Backend source and config: **backend/main.py**, **backend/requirements.txt**, **backend/Dockerfile**, **backend/.dockerignore**, and both backend debug scripts.
- Frontend source and config: **frontend/src** components/hooks/utils, **package.json**, **package-lock.json**, **vite.config.js**, **eslint.config.js**, **postcss.config.js**, **vercel.json**, **index.html**, **index.css**, static SVG assets, and the local **.env** API setting.
- Project docs and deployment files: root **README.md**, **DEPLOY_HF.md**, root **Dockerfile**, GitHub Actions Hugging Face sync workflow, gitignore files, and VS Code settings.
- Data/schema files: every CSV in **f1_data_2025_abudhabi**; no database schema, SQL migration, Prisma schema, docker-compose file, or backend environment example was present.

ML/AI status: no machine-learning model, training script, inference service, embedding pipeline, or evaluation harness exists in this codebase. The project is a data-processing and visualization system, not an ML system.

2. Full Tech Stack Breakdown

The stack is visible directly in **requirements.txt**, **package.json**, **package-lock.json**, the Dockerfiles, and imports inside the source. Python dependencies are unpinned, while npm dependencies are locked by **package-lock.json**.

FastAPI	Backend API in backend/main.py .	The backend is endpoint-centric, JSON-heavy, and benefits from simple path typing such as year: int plus automatic OpenAPI docs.
uvicorn	Backend server for local and Docker startup.	A lightweight ASGI runner fits a single FastAPI service and Hugging Face container deployment.
FastF1	Loads F1 event schedules, race sessions, telemetry, laps, weather, race control, and team radio.	This is the domain-specific data source. Alternatives would require manual Ergast/FIA feed integration, while FastF1 already exposes the timing objects needed.
pandas	Backend telemetry merging, resampling, interpolation, timedelta conversion, and CSV export.	The workload is dataframe-shaped: merge_asof , resample('1s') , interpolate , and to_json are central operations.
requests	Listed in backend requirements; not directly used by current source.	Likely included for HTTP data/debug needs, but currently no application code imports it.
React 19	Frontend UI state, replay controls, lazy-loaded replay engine, memoized derived views.	The app is highly interactive and stateful; useMemo , useEffect , and component composition are used throughout.
Vite	Frontend dev server and production build.	The frontend is an SPA with fast local iteration. vite.config.js also splits bundles into vendor , charts , and d3 chunks.
Axios	Frontend HTTP calls to seasons, races, telemetry replay, and team radio.	The code uses simple promise-based JSON requests and a catch fallback for optional radio data.
D3	Scale calculation, extents, line generation, and min/max operations.	The app needs custom SVG track geometry from telemetry coordinates. D3 is used for math and path generation, while React renders declaratively.
Recharts	Telemetry and gap charts.	Standard responsive line charts are faster to assemble with Recharts than hand-writing every chart axis and tooltip.
Tailwind CSS 4	Utility classes across JSX plus custom tokens in index.css .	The frontend uses dense utility-driven layout and app-specific terminal tokens for colors, fonts, panels, and controls.
PostCSS and autoprefixer	CSS processing pipeline.	Required support tooling for Tailwind and browser-compatible CSS output.
ESLint	Frontend linting with React hooks and refresh rules.	Catches unused variables and hook mistakes in a complex component tree.
Docker	Root and backend Dockerfiles for Python 3.11 slim backend containers.	Hugging Face Spaces uses Docker SDK; the repo keeps a root Dockerfile that copies only backend runtime files.

GitHub Actions	.github/workflows/huggingface.yml syncs main to Hugging Face Hub.	It deploys a clean backend snapshot because Hugging Face rejects large normal-git files over 10 MB.
Vercel	frontend/vercel.json rewrites all routes to index.html .	The frontend is an SPA and needs history fallback on refresh or deep links.

Direct dependency versions from the lockfile

- **react** and **react-dom**: 19.2.1 resolved, package.json allows ^19.2.0.
- **vite**: 7.2.6 resolved, package.json allows ^7.2.4; **@vitejs/plugin-react**: 5.1.1.
- **d3**: 7.9.0; **recharts**: 3.5.1; **axios**: 1.13.2.
- **tailwindcss** and **@tailwindcss/postcss**: 4.1.17; **postcss**: 8.5.6; **autoprefixer**: 10.4.22.
- **eslint**: 9.39.1; **@eslint/js**: 9.39.1; **eslint-plugin-react-hooks**: 7.0.1; **eslint-plugin-react-refresh**: 0.4.24; **globals**: 16.5.0.

3. System Architecture

High-level architecture

- **Client:** a **React** SPA renders the terminal shell, season/race selectors, replay map, leaderboard, controls, charts, strategy panels, and messages.
- **API layer:** a single **FastAPI** application exposes JSON endpoints under **/api** and serves no database-backed writes.
- **Data provider:** **FastF1** loads schedule/session data and stores generated cache artifacts under **f1_cache** or **/tmp/f1_cache** in cloud-style environments.
- **Processing layer:** **pandas** merges irregular FastF1 feeds, resamples telemetry to 1 second, interpolates continuous car/position channels, forward-fills categorical channels, and serializes JSON.
- **Deployment:** backend is containerized for **Hugging Face Spaces**; frontend is Vite/Vercel-friendly and reads **VITE_API_URL**.

End-to-end data flow

- The browser loads **App.jsx**, asks **GET /api/seasons**, merges the API years with the current browser year, and lets the user pick a season.
- When a season is selected, the browser calls **GET /api/{year}/races**. The backend calls **fastf1.get_event_schedule(year)** and returns schedule rows as JSON.
- When a race is selected, **RaceReplay** lazy-loads and **useRaceReplayData** starts two concurrent requests: **team_radio** and full **telemetry_replay**.
- The backend loads the FastF1 race session with **telemetry=True**, **laps=True**, **weather=True**, and **messages=True**, with a fallback for older FastF1 versions that do not accept the messages parameter.
- For each driver, the backend prefers full-session **pos_data** and **car_data**. It normalizes **Time**, uses **pandas.merge_asof** to join position and car channels, and falls back to lap telemetry if full-session data is unavailable.
- Lap metadata is merged backward onto telemetry using **LapStartTime**, then telemetry is resampled at **1 Hz**. Numeric channels such as **X**, **Y**, **Speed**, **Distance**, **Throttle**, **Brake**, and **RPM** are interpolated; categorical fields such as **LapNumber**, **Compound**, **nGear**, and **DRS** are forward-filled.
- The backend computes a global minimum telemetry time, shifts telemetry, laps, events, race control, and weather onto a shared zero-based timeline, and returns a single replay payload.
- The frontend groups telemetry and laps by driver, derives current positions from binary search over sorted telemetry rows, renders an SVG track with **D3** scales and paths, and updates playback with **requestAnimationFrame**.
- Downstream panels gate their data by **currentTime**, so future pit stops, sectors, strategy stints, gaps, and messages do not appear before they should.

API design

GET /	No parameters.	Health-style JSON message confirming the API is running.
GET /api/build_info	No parameters.	Reads build_info.json if present; returns build timestamp/Python version or a note. Used to verify Hugging Face image freshness.
GET /api/seasons	No parameters.	Returns {seasons: [2018..current_year]} .
GET /api/{year}/races	year is typed as int by FastAPI.	Returns FastF1 event schedule rows serialized from pandas.
GET /api/{year}/{race_name}/race/telemetry_replay	year and race_name path params.	Large replay payload with telemetry, drivers, laps, events, race_control, circuit_info, weather, total_laps, and time_base.
GET /api/{year}/{race_name}/race/telemetry_replay_meta	Same race path params.	Lightweight counts/ranges: driver count, total laps, time min/max, presence of car/position data, row counts, expected keys.
GET /api/{year}/{race_name}/race/team_radio	Same race path params.	Returns radio messages with Time, Driver, and Message. On failure, returns an empty list instead of failing the replay.

Data schema and relationships

drivers	DriverNumber, Abbreviation, TeamName, TeamColor, GridPosition, ClassifiedPosition, TotalTime.	Dictionary keyed by driver number; joins UI telemetry, laps, leaderboard, podium, and charts.
telemetry	Time, X, Y, Speed, Distance, Throttle, Brake, RPM, DRS, LapNumber, Compound, Driver.	Many rows per driver. Drives track map, speed/input charts, lap state, and ranking fallback.
laps	Driver, DriverAbbreviation, LapNumber, Position, Stint, Compound, TyreLife, LapTime, LapStartTime, PitInTime, PitOutTime, sector times.	Many laps per driver. Drives strategy, pit feed, sector table, gap chart, fastest lap, and finish timing.
events	Time, Status, Message.	Track status feed for green/yellow/red/safety-car/VSC overlays.
race_control	Time, Category, Message, Flag, Scope, Sector, RacingNumber, Lap.	Normalized into current-time-visible messages and fallback flag detection.
weather	Time, AirTemp, Humidity, Pressure, Rainfall, TrackTemp, WindDirection, WindSpeed.	Binary search picks the latest row at or before replay time.
CSV export folder	race_results, all_laps, telemetry, weather, track_status, race_control_messages, session_info.	Offline example/export from extract_abudhabi_2025.py ; no relational database is used.

Caching, async, and event-driven patterns

- **FastF1 Cache** is enabled at startup. Local development uses **f1_cache**; cloud-style Vercel/Render environments use **/tmp/f1_cache**.

- The checked-out working folder also contains ignored generated cache data: a large FastF1 HTTP SQLite cache and many **.ff1pkl** session artifacts.
- The frontend has an in-memory **replayRequestCache** map keyed by API URL, year, race, and retry token to dedupe heavy full-replay requests, especially under React StrictMode.
- Playback is event-loop driven with **requestAnimationFrame**; there are no backend queues, Celery workers, message brokers, or WebSocket streams.
- GitHub Actions is event-driven on pushes to **main** and manually via **workflow_dispatch**, then force-pushes a clean backend snapshot to Hugging Face Spaces.

4. Key Technical Decisions

Use FastF1 rather than hand-maintained race data

The app needs official-like schedule, results, telemetry, laps, weather, status, race control, and radio objects. FastF1 already models those feeds and exposes session-level cache support. The trade-off is that endpoint behavior depends on FastF1 data availability and version-specific APIs.

Use FastAPI as a JSON processing service

The backend is a small API surface with typed path parameters and simple JSON responses. FastAPI keeps this lightweight and gives OpenAPI verification. The trade-off is that expensive race processing happens inside request/response endpoints rather than a background precompute pipeline.

Use pandas for telemetry shaping

The hard backend work is dataframe alignment: timedeltas, as-of joins, resampling, interpolation, and JSON serialization. Pandas is the right fit for irregular time-series tabular data. A pure Python loop would be harder to reason about and slower to iterate on.

Prefer full-session pos_data plus car_data

The backend avoids truncated lap telemetry by joining full-session position and car channels, then falls back to lap telemetry only when those feeds are missing. The trade-off is more memory and CPU per replay request.

Resample telemetry at 1 second

A 1 Hz replay is smooth enough for visual playback while reducing raw data volume. A higher frequency would make charts and payloads heavier; a lower frequency would make cars jump and charts lose detail.

Use a shared zero-based replay timeline

The backend calculates `global_t0` and shifts telemetry, laps, track status, race control, and weather into the same time domain. This is essential because FastF1 mixes session-relative timedeltas and absolute timestamps.

Do not inject synthetic lap-zero telemetry

A code comment explains that adding a fake time-zero row caused Lap 1 to appear to last through formation/grid delay. The chosen approach leaves leading LapNumber as null until real Lap 1 data exists.

Expose telemetry_replay_meta

Full replay payloads are expensive. A lightweight metadata endpoint lets developers and deployments confirm driver count, lap count, time range, data presence, and expected payload keys without pulling all telemetry.

Render the track declaratively in React SVG using D3 math

D3 computes extents/scales/paths, while React owns DOM rendering. This avoids imperative teardown/redraw loops and fits React state changes.

Use React lazy and Vite manual chunks

The heavy replay component and chart/D3 libraries are split so the shell can load before the replay engine and production bundles are separated by vendor/chart responsibilities.

No database by design

The canonical data comes from FastF1 plus generated cache, and the app is read-only. Avoiding a database keeps the student/demo deployment simpler. The trade-off is no persisted user sessions, no precomputed race warehouse, and repeated expensive generation when cache is cold.

Deploy backend to Hugging Face Spaces

The docs explicitly rejected Render free tier memory limits and chose Hugging Face Spaces for free 16 GB RAM/2 vCPU. That matches heavy FastF1 race processing. The trade-off is container cold start and separate frontend/backend hosting.

Security, auth, validation, and error-handling choices

- **Authentication:** none is implemented. The app is public/read-only and does not store user accounts or private data.
- **CORS:** backend reads **ALLOWED_ORIGINS**, supports comma-separated origins, strips trailing slashes, and disables credentials when origins are wildcard to avoid invalid CORS combinations.
- **Request validation:** FastAPI validates **year** as an integer. Race names are strings from the frontend schedule dropdown. There are no Pydantic request models in active use; **BaseModel** is imported but unused.
- **Data validation:** the backend checks column presence before selecting/transforming, safely converts team colors, coerces numeric telemetry before interpolation, handles datetime/timedelta variants, and serializes through pandas JSON to turn NaN into null.
- **Error handling:** endpoint-level failures become **HTTPException(500)**; per-driver telemetry failures are logged and skipped; radio failures return an empty list; frontend failures show a retry UI.
- **Secrets:** deployment secrets are GitHub Actions secrets such as **HF_TOKEN** and **HF_SPACE_ID**. The frontend **.env** contains a public Vite API URL, which is expected because Vite frontend env values are client-visible.

5. Deep Dive - Hardest Problems

1. Aligning heterogeneous race timing streams

FastF1 exposes telemetry, car channels, laps, weather, race control, track status, and radio in different shapes. Some fields are **Timedelta**, some can be **Timestamp**, and race-control/radio messages can be absolute datetimes. The backend solves this with `_time_column_to_seconds`, explicit timedelta conversions, and `_shift_time_fields` to put everything on the same replay clock.

2. Joining position and car telemetry without corrupting time

The backend sorts each driver's position and car data by **Time** and uses `pandas.merge_asof` with a 250 ms nearest-neighbor tolerance. That is a practical way to combine feeds sampled at slightly different timestamps without requiring exact equality. The code then merges lap metadata backward by lap start time so every telemetry row can carry tyre compound and current lap.

3. Heavy payload latency and duplicate work

A full replay can include hundreds of thousands of telemetry points. The project addresses this with FastF1 cache, 1 Hz resampling, the lightweight `telemetry_replay_meta` endpoint, frontend in-flight request dedupe, memoized groupings, binary search over sorted telemetry, sliced chart windows, and Vite chunk splitting. The remaining trade-off is that the full replay endpoint is still expensive on cold cache.

4. Correct replay visibility

The app avoids the common replay bug of showing future information. **StrategyPanel**, **PitStopAnalysis**, **SectorTimes**, **GapChart**, **RaceMessagesPanel**, and fastest-lap logic all gate rows against **currentTime**. That means pit stops, sectors, race messages, and gap data only appear once the replay clock has reached them.

5. Leaderboard correctness during race start, race progress, and finish

RaceReplay.jsx handles different ranking modes. At race start it respects grid position, during the race it sorts by lap and distance, and after the official finish it uses **ClassifiedPosition**. It freezes final standings once **raceEndTime** or replay end is reached, then shows a podium overlay from official standings.

6. Rendering a custom track map without imperative redraw churn

The track map builds a D3 line path from valid telemetry coordinates, scales it into an **800 x 500** SVG viewBox, and lets React render paths and driver markers. Driver positions are found with binary search and interpolated between nearby samples if the gap is under 2 seconds.

7. Deployment constraints around large race data

The repository includes a large checked-in telemetry CSV and a much larger ignored FastF1 cache. The Hugging Face workflow deliberately builds a clean deploy snapshot containing only backend runtime files and the root Dockerfile, because Hugging Face rejects large normal-git files over 10 MB.

What would be hardest to scale next

- Moving replay generation out of synchronous request handling into precompute jobs or cached race JSON artifacts.

- Adding server-side compression, pagination, or streaming for telemetry payloads.
- Pinning backend dependency versions to avoid FastF1/pandas behavior drift.
- Adding integration tests around time normalization, replay shape, and frontend current-time gating.
- Introducing an object store or database if the app needs user-created sessions, saved analyses, or precomputed race catalogs.

6. Interview Questions and Answers

Q1. Walk me through the architecture of GridPulse.

Answer: GridPulse has a **React/Vite** SPA frontend and a **FastAPI** backend. The frontend first calls **/api/seasons** and **/api/{year}/races**, then lazy-loads the replay engine once a race is selected. The backend uses **FastF1** to load race sessions and **pandas** to align position data, car telemetry, laps, weather, track status, race control, and radio. The full replay endpoint returns a single time-normalized JSON payload. The browser groups data by driver, advances **currentTime** with **requestAnimationFrame**, and renders a D3-derived SVG track, leaderboard, telemetry charts, strategy panels, and race messages.

Concept tested: System architecture and data flow.

Q2. Why did you use FastF1 and pandas instead of building the data layer manually?

Answer: **FastF1** is the domain-specific source for F1 schedules, sessions, telemetry, laps, weather, race control, and radio. Rebuilding that from generic APIs would be much harder and less complete. **pandas** fits the transformation workload because the backend needs **merge_asof**, resampling, interpolation, timedelta conversion, dataframe filtering, and JSON serialization. The trade-off is heavier memory usage and dependency on FastF1 data availability, but the chosen stack directly matches the shape of the problem.

Concept tested: Framework/library choice and trade-offs.

Q3. How does the backend turn raw FastF1 data into replay-ready JSON?

Answer: For each driver, the backend loads full-session **pos_data** and **car_data**, normalizes **Time** to timesteps, and joins the feeds with **pandas.merge_asof** using a 250 ms tolerance. It merges lap metadata backward by **LapStartTime**, resamples to **1 second**, interpolates continuous columns like **X**, **Y**, **Speed**, **Distance**, **Throttle**, **Brake**, and **RPM**, and forward-fills categorical fields like **Compound**, **LapNumber**, **nGear**, and **DRS**. It then shifts all streams onto a shared zero-based timeline and returns JSON.

Concept tested: Data processing and time-series correctness.

Q4. What were the biggest correctness risks in the replay UI?

Answer: The largest risk is showing data before it actually happened. The code handles this by gating pit stops, sectors, gap chart points, fastest lap, strategy rows, race control, and team radio by **currentTime**. Another risk is leaderboard order: the app uses grid order near race start, lap/distance during the race, and official **ClassifiedPosition** at finish. It also avoids a known bad synthetic time-zero row because that made Lap 1 include formation delay.

Concept tested: Replay correctness and temporal reasoning.

Q5. How did you address performance with such a large telemetry payload?

Answer: The backend reduces raw data by resampling to 1 Hz and caches FastF1 data. It also provides **telemetry_replay_meta** so deployments can check race shape without downloading the full payload. The frontend caches in-flight replay requests, groups telemetry/laps by driver with **useMemo**, uses binary search to find each driver's current sample, slices only a 60-second chart window, and splits Vite bundles into vendor/chart/D3 chunks.

Concept tested: Latency, client performance, and scalability.

Q6. Why is there no database schema or migration layer?

Answer: The current app is read-only and its canonical data source is external **FastF1** session data plus generated cache files. There are no user accounts, saved analyses, or write flows, so a database would add operational complexity without solving the current product problem. The trade-off is that cold replay generation can still be expensive and there is no persistent precomputed warehouse. If this scaled, I would consider storing precomputed replay artifacts in object storage or a database.

Concept tested: Database design and scope control.

Q7. What is the API contract for the replay endpoint?

Answer: **GET /api/{year}/{race_name}/race/telemetry_replay** returns a JSON object with **telemetry**, **drivers**, **laps**, **events**, **race_control**, **circuit_info**, **weather**, **total_laps**, and **time_base**. Telemetry rows are keyed by driver and time. Driver info is keyed by driver number. Laps include tyre, pit, sector, and lap timing fields. The frontend assumes those top-level keys and has defensive array/object checks before using them.

Concept tested: API design and data contracts.

Q8. How is error handling implemented?

Answer: Backend endpoint failures are wrapped in **HTTPException(status_code=500)**. Inside the heavy telemetry loop, per-driver exceptions are logged and skipped so one bad driver's data does not kill the entire response. The team-radio endpoint returns an empty list on failure because radio is optional. On the frontend, **useRaceReplayData** catches replay load failure, shows a user-facing retry state, and keeps the UI from rendering partial invalid data.

Concept tested: Resilience and graceful degradation.

Q9. What security and authentication model does the app have?

Answer: There is no authentication because the current app is a public, read-only race analytics viewer. Backend security is mainly CORS configuration: **ALLOWED_ORIGINS** can be wildcard or comma-separated origins, and credentials are disabled when wildcard origins are used. Deployment secrets such as **HF_TOKEN** and **HF_SPACE_ID** live in GitHub Actions secrets. The Vite API URL is public client config, not a private secret.

Concept tested: Security model and deployment configuration.

Q10. How would you scale this beyond a demo or student project?

Answer: I would precompute replay payloads per race, store them in object storage or a database, serve compressed JSON or binary chunks, and invalidate when FastF1 data changes. I would add backend dependency pinning, integration tests for replay shape and time alignment, HTTP caching headers, and possibly a background job queue for cold race generation. For the frontend, I would virtualize heavy tables if needed and load telemetry in chunks by time window or driver.

Concept tested: Scalability and productionization.

Q11. Why did the project choose Hugging Face Spaces for backend deployment?

Answer: The deployment doc explicitly says Render's free 512 MB tier is too small for full F1 race processing, while Hugging Face Spaces offers a free Docker environment with 16 GB RAM and 2 vCPU. That matches the backend's heavy FastF1/pandas workload. The root Dockerfile copies backend requirements and code, writes **build_info.json**, creates **f1_cache**, exposes port 7860, and starts unicorn.

Concept tested: Infrastructure decision-making.

Q12. What would you improve first if you had another week?

Answer: I would pin backend dependency versions, add tests around `_time_column_to_seconds`, `_shift_time_fields`, replay endpoint shape, and frontend gating utilities, then add precomputed/cached replay artifacts so the full endpoint does not have to rebuild heavy payloads on demand. I would also replace broad prints with structured logging and remove unused template artifacts like `App.css`, unused imports, and unused requirements.

Concept tested: Engineering judgment and maintainability.

7. Follow-up Questions

Q1. What happens if FastF1 changes a column name or returns missing data?

Answer: The backend already checks for column presence in many places, uses fallbacks for older FastF1 load signatures, and skips failed drivers. The next step would be explicit contract tests using cached fixture sessions, clearer 502/503 errors for upstream data issues, and dependency pinning so behavior changes are intentional.

Concept tested: Dependency drift and defensive coding.

Q2. Why not use WebSockets for replay?

Answer: This is a post-race replay, not a live ingestion system. The backend returns a complete replay payload and the browser controls time locally. WebSockets would add complexity without reducing the expensive initial data shaping. If the product became live race monitoring, WebSockets or server-sent events would become more relevant.

Concept tested: Choosing the right communication pattern.

Q3. How do you know the leaderboard is not just sorting by track distance incorrectly?

Answer: The code uses different modes. It respects grid position near start, then sorts by current lap and distance during the race, uses cumulative completed lap time as a backup, and switches to official **ClassifiedPosition** at race finish. This avoids using raw distance as the only truth across pits, retirements, and finish states.

Concept tested: Correctness beyond naive sorting.

Q4. What is the main bottleneck today?

Answer: The main bottleneck is cold generation of the full **telemetry_replay** payload: FastF1 session loading, per-driver merges, resampling, interpolation, JSON serialization, and network transfer. Frontend rendering has optimizations, but backend precompute and payload strategy would give the biggest production gains.

Concept tested: Performance diagnosis.

Q5. How would you test the time-gating logic?

Answer: I would unit-test **replayMath.js** with lap, sector, pit, and message fixtures around edge times before, at, and after event timestamps. Then I would add an integration fixture where a known pit stop, sector time, and race-control message must be absent before `currentTime` and present after `currentTime`.

Concept tested: Testing strategy.

Q6. What makes the project more than a CRUD app?

Answer: It has no CRUD surface. The interesting engineering is time-series alignment, large payload control, replay-clock correctness, domain-specific F1 semantics, custom SVG track rendering, and a dense analytics UI with multiple synchronized derived panels.

Concept tested: Project positioning and technical depth.

8. Quick Reference Card

Elevator pitch

GridPulse is a full-stack Formula 1 replay analytics terminal. I built a **FastAPI** and **FastF1** backend that converts raw race timing, telemetry, lap, weather, race-control, and radio feeds into one synchronized replay payload, and a **React** frontend that lets users scrub through a race with a live track map, leaderboard, telemetry charts, tyre strategy, pit stops, sectors, weather, and final classification. The technically interesting part is making large, irregular time-series data correct, fast enough, and easy to inspect in the browser.

Five most impressive technical facts

- The backend joins **position data** and **car telemetry** with **pandas.merge_asof** and a 250 ms tolerance.
- Telemetry is resampled to **1 Hz**, numeric channels are interpolated, and categorical channels are forward-filled for smooth replay.
- All streams are shifted to a shared zero-based timeline using a global telemetry offset, solving mixed timedelta/datetime alignment.
- The frontend uses request dedupe, memoized driver groupings, binary search, 60-second chart windows, and Vite chunk splitting to control large replay costs.
- Replay panels deliberately gate future data by **currentTime**, so pit stops, sectors, gaps, strategy, and messages appear only when they should.

Three ways it stands out from typical student projects

- It is a domain-specific analytics system, not a CRUD dashboard: the hard problem is synchronized motorsport telemetry.
- It handles real deployment constraints: Hugging Face Docker backend, Vercel SPA config, GitHub Actions sync, build freshness endpoint, and large-file deployment avoidance.
- It includes real performance and correctness thinking: metadata endpoint, cache strategy, frontend request dedupe, chunk splitting, and current-time visibility rules.

One-minute technical talking path

- Start with the problem: raw F1 race data is high-volume and split across timing, car, position, weather, control, and radio feeds.
- Explain the backend: FastF1 loads sessions, pandas aligns/resamples/interpolates, FastAPI returns replay JSON.
- Explain the frontend: React owns replay state, D3 creates track geometry, Recharts draws telemetry/gap views, and panels derive only what is visible at currentTime.
- Close with scale: the next production step is precomputing replay artifacts and adding tests around timeline normalization and replay contracts.